

Deep Limit Order Book Forecasting

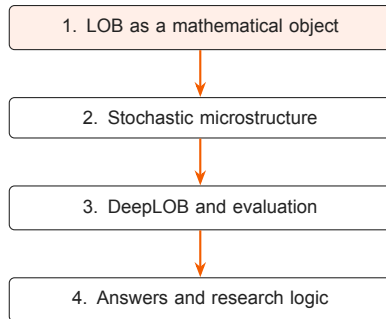
A Microstructure-First Introduction

Yida Ding May 27, 2026

■ Opening Questions

Keep these in mind

- ▶ What exactly is a LOB forecasting model predicting?
- ▶ Why do tick size, spread, and horizon change the difficulty?
- ▶ Why can a classifier be accurate but still be weak as a trading signal?



■ The Object: A Queue-Shaped State

- ▶ A LOB records resting buy and sell orders that have not yet traded.
- ▶ Each level has a price and a queue size: best ask, second ask, best bid, second bid, and so on.
- ▶ Deep LOB models usually keep the first $L = 10$ levels and write one snapshot as a vector.

One Snapshot

$$L(\tau) = \{p_\ell^{ask}, v_\ell^{ask}, p_\ell^{bid}, v_\ell^{bid}\}_{\ell=1}^L \in \mathbb{R}^{4L}.$$

The Model's Observation

$$X_\tau = (L(\tau - T + 1), \dots, L(\tau))$$

$$X_\tau \in \mathbb{R}^{T \times 4L}$$

What to Picture

The input is not a price line. It is a short movie of queue shapes on both sides of the market.

■ A Queueing Model of the Book

- ▶ A useful abstraction is a continuous-time state process:

$$Q_t = (Q_t^{b,1}, \dots, Q_t^{b,L}, Q_t^{a,1}, \dots, Q_t^{a,L}).$$

- ▶ Each coordinate is queue size at one bid/ask level.
- ▶ Limit orders, market orders, and cancellations are jumps of Q_t .

Point-process view

$$dQ_t = \sum_{e \in \mathcal{E}} \nu_e(Q_{t-}) dN_t^e$$

$$\mathbb{P}(dN_t^e = 1 \mid \mathcal{F}_{t-}) \approx \lambda_e(Q_{t-}) dt$$

Mathematical Meaning

The book is not a smooth curve; it is a queueing system hit by random order-flow events.

■ Price Changes as Queue Depletion

Price moves when a queue is depleted

$$\tau_a = \inf\{t : Q_t^{a,1} = 0\}$$

$$\tau_b = \inf\{t : Q_t^{b,1} = 0\}$$

$$\text{next move} = \begin{cases} \text{up,} & \tau_a < \tau_b, \\ \text{down,} & \tau_b < \tau_a. \end{cases}$$

- ▶ Queue imbalance is predictive because it changes depletion probabilities.

$$I_t = \frac{Q_t^{b,1} - Q_t^{a,1}}{Q_t^{b,1} + Q_t^{a,1}}.$$

- ▶ Event clustering matters: order arrivals are often self- and cross-exciting.

$$\lambda_i(t) = \mu_i + \sum_j \int_0^t \phi_{ij}(t-s) dN_j(s).$$

- ▶ Forecasting asks for conditional probabilities such as

$$\mathbb{P}(\tau_a < \tau_b \mid \mathcal{F}_t).$$

■ From Queue Dynamics to a Label

Mid-Price and Spread

$$m_\tau = \frac{p_1^{ask}(\tau) + p_1^{bid}(\tau)}{2}$$

$$\sigma_\tau = p_1^{ask}(\tau) - p_1^{bid}(\tau)$$

- ▶ m_τ : the reference price used to define direction.
- ▶ σ_τ : the bid–ask spread, also a direct trading friction.

Three-Class Label

$$y_\tau = \begin{cases} -1, & m_{\tau+\Delta\tau} - m_\tau \leq -\theta, \\ 0, & |m_{\tau+\Delta\tau} - m_\tau| < \theta, \\ +1, & m_{\tau+\Delta\tau} - m_\tau \geq +\theta. \end{cases}$$

- ▶ θ : tick size, the smallest admissible price change.
- ▶ $\Delta\tau$: a horizon in future LOB updates, not in fixed seconds.

■ The Equation We Would Like to Solve

A stochastic model gives the target

The queue model says that price direction is a conditional probability:

$$\mathbb{P}(y_\tau = c \mid X_\tau), \quad c \in \{-1, 0, +1\}.$$

Equivalently, at the best quotes:

$$\mathbb{P}(\tau_a < \tau_b \mid \mathcal{F}_t).$$

What closed form would require

- ▶ Known event intensities $\lambda_e(Q_t)$.
- ▶ A small enough state space for dynamic equations.
- ▶ Stable parameters over time and across stocks.
- ▶ Tractable dependence among order arrivals, trades, and cancellations.

Why this is hard in real data

Ten levels, intraday non-stationarity, event clustering, and tick-regime effects make the exact transition law too large and too unstable to solve directly.

■ Learning as Conditional Probability Estimation

Same target, different route

The stochastic model defines the object:

$$\mathbb{P}(y_\tau \mid X_\tau).$$

The neural network supplies a flexible approximation:

$$f_\omega(X_\tau) \approx \begin{pmatrix} \mathbb{P}(y_\tau = -1 \mid X_\tau) \\ \mathbb{P}(y_\tau = 0 \mid X_\tau) \\ \mathbb{P}(y_\tau = 1 \mid X_\tau) \end{pmatrix}.$$

Maximum likelihood

Cross-entropy is not just an engineering loss. It is maximum likelihood for the categorical direction label:

$$\min_{\omega} - \sum_{\tau} \log f_{\omega}(X_{\tau})_{y_{\tau}}.$$

Connection

Stochastic modeling tells us what probability to estimate; machine learning estimates it without forcing a closed-form transition law.

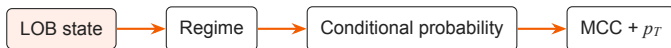
■ The Microstructural Guide's Test

Empirical setup

- ▶ 15 NASDAQ stocks from 2017–2019.
- ▶ Full message-level LOBSTER order-book data.
- ▶ Input: the last 100 LOB updates and the first 10 levels.
- ▶ Output: direction after H_{10}, H_{50}, H_{100} .

Conceptual question

- ▶ Which market regimes make the conditional distribution easier to learn?
- ▶ Does good classification imply a usable trading path?
- ▶ Are results explained by model architecture, or by microstructure?



LOBFrame as a Controlled Experiment

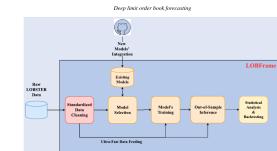


Figure 2: Partial representation of LOBFrame. This framework facilitates the Limit Order Book (LOB) forecasting practice through a standard pipeline that includes data cleaning, standardized preprocessing, model selection, and integration of both trading and neural forecasting models, and ultra-fast data feeding. It enables comprehensive model training, out-of-sample inference, statistical analysis, and robust backtesting, offering the academic and practitioner communities a benchmark tool for advancing research and standardizing practices in the field. The size of distinct colors in the diagrams highlights logically separated stages of the pipeline.

assumption. Indeed, weights are learned during inference, and derived features (i.e. learned from the training set) are data adaptive. The LSTM layer, on the other side, is used to capture residual time dependencies among the (training) features. It is worth underlining that short time dependencies are already captured by the convolutional layers, which take LOB snapshots as inputs (see Figure 3). To train, validate and test the DeepLOB model, we design a high performance data loader, which samples mini-batches of size 32 (as per in the original model's implementation), each made of inputs with size 100×40 . Dimension 100 (i.e. the temporal dimension) represents the history length and corresponds to the number of historically consecutive LOB updates constituting each sample. Dimension 40, instead, is the number of spatial constraints for each LOB's snapshot (see equation (1)). The sampling process differs for the training, validation, and test sets. During training, the tick-sampling is random and balanced. From each trading day, we select the number of samples for the less represented class and (i) if this value is ≥ 5000 , then we sample 5000 random representation (representative is a 100×40 inputs) for each of the three classes (see equation (2)), otherwise, (ii) if this value is < 5000 , then we sample a number of random representatives for each class which is equal to the number of samples for the less represented class.

During validation and test stages, we still sample batches with a size of 32, but they are always sequential and cover the totality of data in the two sets. In line with the related literature (Zhang et al., 2019), the model is always trained for a maximum number of epochs equal to 100, with patience equal to 15 epochs. We use a modified version of the Adam optimizer (Kingma and Ba 2014) with decoupled weight decay (Loshchilov and Hutter 2017), commonly known as "AdamW". Following the latest applied research findings

(Ilieva et al., 2020; Karpathy 2024), we use a learning rate equal to 6×10^{-5} , a β_1 decay rate equal to 0.9 and a β_2 decay rate equal to 0.95. The choices of values for these parameters are determined by the training pipeline described above, which is different from the one proposed in the original work (Zhang et al., 2019) and relies on a smaller number of training samples to reduce the model's exposition to the noise characterizing the LOB. The entire framework described in this paper is coded in Python using the PyTorch deep learning library (Paszke et al., 2019). A total number of 135 experiments have been run on the University College London Computer Science Department's High-Performance Computing Cluster (UCL 2024) for a cumulative GPU runtime of 999 hours, 16 minutes and 27 seconds. Six different types of GPUs have been used: (i) NVIDIA GeForce GTX 1080 Ti; (ii) NVIDIA GeForce RTX 2080 Ti; (iii) NVIDIA TITAN X (Pascal); (iv) NVIDIA TITAN Xp; (v) Tesla V100 PCIe 16GB; and (vi) Tesla V100 PCIe 32GB.

6. Microstructural prices

In this section, we investigate stocks' fundamental microstructural properties, focusing on the interplay between tick size and bid-ask spread, liquidity at the best levels, and the structural organization of the Limit Order Book (LOB). Through this analysis:

- We identify a practical classification standard for tick-based stock categories, identifying unique behavioral patterns across small-, medium-, and large-tick stocks and revealing the macro-stability of spread distributions over the 3-year analysis period.

Source: Briola et al., "Deep Limit Order Book Forecasting: A Microstructural Guide", Figure 2.

■ Tick Size Sets the Scale

Classify by spread relative to tick

$$\bar{\sigma} = \mathbb{E}[\sigma_\tau], \quad r = \frac{\bar{\sigma}}{\theta}$$

small-tick	$\bar{\sigma} > 3\theta,$
medium-tick	$1.5\theta < \bar{\sigma} < 3\theta,$
large-tick	$\bar{\sigma} < 1.5\theta.$

- ▶ Large-tick stocks often have spreads near one tick, so best-level queues are central.
- ▶ Small-tick stocks can have wider spreads and more dispersed liquidity.
- ▶ The same label rule therefore creates different learning problems across stocks.

Scale of a Price Move

Tick size is not only an exchange rule; it sets the economic meaning of “one move” in that market.

■ DeepLOB as a Conditional-Probability Machine

JOURNAL OF FINANCIAL MARKETS, VOL. 50, NO. 00, 2020

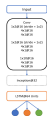


Figure 3. Model architecture schematic. Flow (a) (b) (c) represents a complete limit order book at time t of size $n \times m$. (d) represents the output of the model and (e) represents the output of the model.

signal processing [33]. FIR filters are popular smoothing techniques for denoising input signals and they are simple to implement and work well. We can write any FIR filter in the following form:

$$y(n) = \sum_{k=0}^{N-1} h_k x(n-k) \quad (5)$$

where the output signal $y(n)$ at any time n is a weighted sum of a finite number of past values of the input signal $x(n)$. The filter order is denoted as N and h_k is the filter coefficients. In a convolutional neural network, the coefficients of the filter kernel are not obtained via a statistical objective from the training data but are instead learned via a statistical objective from the training data. The network orders are used to estimate the value function at output.

The details of the first convolutional layer operation is used kernel to "scan" through input data, the layers of limit order book information is used. Recall that we take the most 100 recent orders of an order book to form a single input and there are 40 features per time stamp, so the size of a single input is (100×40) . We organize the 40 features as following:

$$[b_1^{(t)}, b_2^{(t)}, \dots, b_{100}^{(t)}, v_1^{(t)}, v_2^{(t)}, \dots, v_{100}^{(t)}] \quad (6)$$

where i denotes the i -th level of a limit order book. The size of our first convolutional filter is (1×2) with stride of (1×2) . The first layer essentially summarizes information between price and volume $\{p^{(t)}, v^{(t)}\}$ at each order book level. The usage of stride is necessary here as an important property of convolutional layers is parameter sharing. This property is attractive as few parameters are needed, largely avoiding overfitting problems. However, without strides, we would apply some parameters to $\{p^{(t)}, v^{(t)}\}$ and $\{p^{(t)}, v^{(t+1)}\}$. In other words, $p^{(t)}$ and $v^{(t)}$ would share some parameters

because the kernel filter moves by one step, which is obviously wrong as price and volume have different dynamic behaviors.

Because the first layer only captures information at each order book level, we would expect representative features to be extracted when integrating information across multiple order book levels. We can do this by adding another convolutional layer with filter size (1×2) and stride (1×2) . The resulting feature maps actually form the micro-price defined by [33]

$$p^{micro-price} = \frac{1}{2} \left(p^{(t)} + p^{(t+1)} \right) \quad (7)$$

The weight $\frac{1}{2}$ is called the imbalance. The micro-price is an important indicator as it considers volume on bid and ask side, and the imbalance between bid and ask side is a very strong indicator of the next price move. This feature of imbalance has been reported by a variety of researchers [36,37,38,39]. Unlike the micro-price where only the first order book level is considered, we also consider volume from other prices for all levels of a LOB so the resulting feature maps are of size $(100, 10)$ after two layers with strides. Finally, we integrate all information by using a large filter of size (1×10) and the dimension of our feature maps before the Inception Module is $(100, 1)$.

We apply core padding to every convolutional layer so the time dimension of our input does not change and Leaky Rectifying Linear Units (Leaky-ReLU) [40] are used as activation functions. The hyper-parameter that would gradient when the unit is not active of the Leaky-ReLU is set to 0.01, evaluated by grid search on the validation set.

Another important property of convolution is that of equivariance to translation [41]. Specifically, a function $f(x)$ is equivariant to a function g if $f(g(x)) = g(f(x))$. For example, suppose that there exists a main classification feature f located at (x_{max}, y_{max}) of an image $I(x, y)$. If we shift every pixel of I one unit to the right, we get a new image I' where $I'(x, y) = I(x-1, y)$. We can still obtain the main classification feature of I' and $x_{max} = x_{max} + 1$, while the location of y_{max} will be at $(x_{max}, y_{max}) = (x_{max} - 1, y_{max})$. This is important to information flow, because convolution can find universal features that are decisive to final output. In our case, suppose a feature that enables imbalance is obtained at time t . If the same event happens later at time t' in the input, the exact feature can be extracted later at t' .

We do not use any pooling layer except in the Inception Modules. Although pooling layers help to find representative features in translation of the input, the smoothing nature of pooling can cause underfitting. Common pooling layers are designed for image processing tasks, and they are most powerful when not only can it confirm features exist in the input, instead of where they exist [42]. Time-series data has different characteristics from images, and the location of representative features is important. Our experiences show that pooling layers in the convolutional layers, at least, cause underfitting problems in the LOB data. However, we think pooling is important and new pooling methods should be designed to process time-series data as it is a promising

- ▶ Input X_T : recent snapshots of the queueing system.
- ▶ CNN: local filters for bid–ask imbalance, queue depth, and depth slope.
- ▶ Inception: several filter widths, i.e. several local time scales.
- ▶ LSTM: summarizes path dependence in recent order-flow history.

Output

Source: Zhang, Zohren and Roberts,

$$\text{softmax}(h_w(X_T))_c \approx \mathbb{P}(y_T = c \mid X_T)$$

"Deep Learning for Limit Order Books", Figure 3.

■ Why These Layers Fit the Book

Convolution and Scale

Parameter sharing says: the same local pattern can matter at many recent times.

$$z_{t,k} = \phi \left(\sum_{a,b} W_{a,b,k} X_{t+a,b} \right)$$

Inception repeats this idea with several window widths.

Recurrence

The state is not Markov after compression; recent event history still matters.

$$h_t = \Psi_{\omega}(z_t, h_{t-1})$$

LSTM is a learnable memory for this missing path information.

Why the Architecture Is Reasonable

DeepLOB is a numerical approximation to the conditional probabilities suggested by queueing and point-process models.

■ MCC: Correlation Instead of Raw Accuracy

- ▶ LOB labels are often imbalanced; the stable class may dominate.
- ▶ Accuracy can look good by mostly predicting the majority class.
- ▶ MCC treats prediction and truth as correlated label variables.
- ▶ Random prediction is near 0; perfect prediction is 1.

Confusion-matrix view

$MCC = F(\text{all true/false positives and negatives})$

It does not only ask how often the model is right; it asks how errors are distributed across classes.

Interpretation

The score measures model quality under a specific market regime, not model quality in isolation.

■ Tick Regime Changes the Score

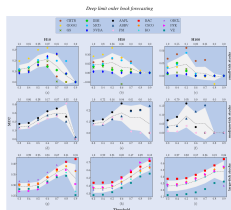


Figure 10: Average Mahalanobis Correlation Coefficient (MCC). Results are reported according to the prediction horizon taken into account (two columns) and stocks' tick size (two rows). Each plot contains three main pieces of information: (i) the model's performance changes applying different thresholds on the probability associated with each forecast (shown on the bottom of the x-axis); (ii) the average percentage amount of winning data (blue line) using the threshold (shown on the top of the x-axis); (iii) the performance average metric and the corresponding standard deviation (shown through the grey line and shadows). All the average values and the standard deviation are computed by considering stocks with the same tick size, spanning the 5-year analysis period.

has a value of 0.078. Differently from what happens for the other two classes of stocks, a clear average growing pattern is detected for threshold values ≤ 0.5 , while the average percentage of data used to compute the metric decreases more than what happened at 0.50, remaining, however, higher than the metrics needed for small- and medium-sized stocks. Overall, large-risk stocks demonstrate greater forecast stability across horizons, as evidenced by the lower variance in MCC. This robustness is strongly linked to their higher liquidity and less frequent spread distributions, which provide a more stable market dynamics for the DeepLOB model to learn from.

The analysis reported in this section is further deepened in Appendixes 2 and 3, where we report (i) the year-wise MCC of the DeepLOB model at H_{10} (H_{50} , H_{100}) for different confidence levels; (ii) the corresponding statistical significance; and (iii) a replica of the analysis in Figure 10 for the FI and accuracy score. The relevance of the results bolsters the robustness of the findings discussed earlier in this Section, highlighting that large-risk stocks exhibit a significant predictability rate across all the considered horizons.

This is evidenced in Figures A1 and A2, where we observe that these stocks achieve FI score realizations greater than 0.45 without applying any thresholds and surpass 0.7 when probability thresholds are implemented. Similarly, accuracy scores exceed 0.7 without thresholds and reach over 0.9 with the application of probability thresholds. However, as detailed in section 7.2, achieving high scores on these traditional machine learning metrics does not necessarily translate into the generation of actionable trading signals. This distinction underlines the complexity of converting predictive accuracy into practical trading strategies.

7.2. On the predictability of model's forecasts

The analysis presented above offers significant insights into the DeepLOB model's performance at different horizons for different classes of stocks. However, further discussion is needed to understand the results from the perspective of the microstructural properties of the LOB. To do so, in this Section, we introduce a novel methodology to evaluate the

- ▶ Rows: small-, medium-, and large-tick stocks.
- ▶ Columns: H_{10}, H_{50}, H_{100} .
- ▶ Horizontal axis: keep only higher-confidence predictions.
- ▶ Reading: performance depends strongly on stock microstructure, not only on the network.

Source: Briola et al., Microstructural Guide,

■ p_T : From Labels to Trading Paths

- ▶ F1/MCC are pointwise: each timestamp is scored separately.
- ▶ Trading is pathwise: entry, holding, and exit must line up.
- ▶ An error near an exit point is more costly than an error in a flat region.

Complete-trade probability

P_T = predicted trade set,

T_T = true trade set.

$$p_T = \frac{|P_T \cap T_T|}{|P_T \cup T_T|}$$

Path-Level Meaning

p_T asks whether the predicted sequence forms the right complete trade, not just whether isolated labels are correct.

■ Microstructure Explains the Scores

1. LOB predictability is jointly determined by the model, label, horizon, and tick regime.
2. Large-tick stocks are often easier: spreads are tight, best queues are thick, and queue changes carry more information.
3. Classification metrics are useful but incomplete; trading signals must also be read as time-ordered paths.
4. The paper's contribution is to put deep forecasting back inside market microstructure.

Reading Rule

Read each score together with the market structure that produced it.

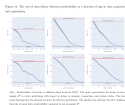
■ Where Predictability Comes From

Ait-Sahalia et al.

- ▶ High-frequency returns and directions do contain short-lived predictability.
- ▶ Latency quickly consumes the value of the signal.
- ▶ Role here: explain why LOB forecasting lives at very short horizons.

DeepLOB

- ▶ A standard CNN + Inception + LSTM architecture.
- ▶ Role here: provide the baseline used by the main paper.
- ▶ Limitation: architecture alone does not solve transferability or tradability.



■ Structure and Generalization

HLOB

- ▶ DeepLOB emphasizes local convolutional structure.
- ▶ HLOB uses a graph to encode dependence among LOB variables.
- ▶ Role here: show how market structure can enter the model design.

Benchmark Study

- ▶ A high score on FI-2010 does not guarantee real-market transfer.
- ▶ Model ranking changes with dataset and horizon.
- ▶ Role here: motivate cross-stock and cross-market validation.

Table 2: Benchmark performance and portfolio return for various models. The table shows performance metrics for different models across various datasets and horizons. The models include DeepLOB, HLOB, and others. The performance is measured in terms of Sharpe Ratio, Maximum Drawdown, and Portfolio Return. The table is divided into two main sections: 'Performance on FI-2010' and 'Performance on real-market data'.

Source: "LOB-Based Deep Learning Models: Benchmark Study", Table 2.

■ The Full Argument

Layer	Question	Papers
Signal	Is there short-horizon predictability, and how fast does it decay?	Ait-Sahalia et al.
State	Is the LOB a price series or a queue-shaped state process?	Cont-de Larrard; Abergel–Jedidi
Approximation Regime	How do we estimate $\mathbb{P}(y_\tau \mid X_\tau)$ in high dimension?	DeepLOB
	How do tick size, spread, and depth change the problem?	Microstructural Guide
Usefulness	Does a good score produce a coherent trading path?	Microstructural Guide; Benchmark Study

■ A Coherent Workflow

1. Define the random object: state Q_t , observation X_τ , and label y_τ .
2. Explain the market regime: $\bar{\sigma}/\theta$, best-queue depth, imbalance, event intensity.
3. Choose a model bias that matches the object: local convolution, recurrence, graph structure, or point-process intensity.
4. Evaluate two levels: sample-wise correctness and path-wise tradability.
5. Test whether the explanation survives across stocks, horizons, and market regimes.

Working Principle

Model complexity should serve a microstructure explanation; it should not replace one.

■ Answers to the Opening Questions

1. What is being predicted?

The future direction of the mid-price, usually after a fixed number of LOB updates:

$$y_\tau = g_\theta(m_{\tau+\Delta\tau} - m_\tau).$$

2. Why do tick size, spread, and horizon matter?

They change queue depletion probabilities, class balance, signal lifetime, and transaction-cost meaning.

3. Why is high accuracy not enough?

A trading signal is a time-ordered path. MCC/F1 count point errors, while p_T asks whether the predicted path completes the right trade.

Closing Claim

The central question is which directional signal is predictable under this market structure, at which horizon, and whether it forms a complete trading path.

■ References and Figure Sources

- ▶ A. Briola, S. Bartolucci, and T. Aste, “Deep Limit Order Book Forecasting: A Microstructural Guide,” Quantitative Finance, 2025.
- ▶ Z. Zhang, S. Zohren, and S. Roberts, “Deep Learning for Limit Order Books,” IEEE Transactions on Signal Processing, 2020.
- ▶ Y. Ait-Sahalia, I. Kalnina, and D. Xiu, “How and When are High-Frequency Stock Returns Predictable?”
- ▶ R. Cont and A. de Larrard, “Price Dynamics in a Markovian Limit Order Market,” SIAM Journal on Financial Mathematics, 2013.
- ▶ F. Abergel and A. Jedidi, “A Mathematical Approach to Order Book Modeling,” International Journal of Theoretical and Applied Finance, 2013.
- ▶ E. Bacry, I. Mastromatteo, and J.-F. Muzy, “Hawkes Processes in Finance,” Market Microstructure and Liquidity, 2015.
- ▶ A. Briola et al., “Information Persistence and Structure in Limit Order Books.”
- ▶ “LOB-Based Deep Learning Models: Benchmark Study.”

Figures are reproduced from the cited papers for internal research discussion; each figure slide lists its source.